

CollisionSplatting: Collision-Aware Motion Planning in 3DGS Scenes with Image-Conditioned Objectives and Adjustable Conservatism

R. Khorrambakht¹, Joaquim Ortiz-Haro¹, Stephan Weiss³, and Ludovic Righetti^{1,2}

Abstract—Incorporating dense visual information into motion planning remains challenging, as geometric planners rely on abstracted scene representations that discard visual richness, while learned visual models often lack geometric interpretability and computational efficiency. This paper introduces CollisionSplatting, a simple, modular, GPU-accelerated, probability-inspired distance metric with tunable conservatism that operates directly on standard 3D Gaussian Splatting (3DGS) scenes. When combined with learned image-conditioned reward functions, this metric enables joint geometric and visual planning by unifying collision-aware costs with image-space objectives. We integrate the metric into GPU-accelerated Model Predictive Path Integral (MPPI) and Rapidly-Exploring Random Tree (RRT) planners, and show on-par or better collision-classification performance compared to representative baselines while achieving substantially higher collision-checking throughput and significantly lower VRAM usage. Finally, we demonstrate the effectiveness of our metric in real-world vision-guided navigation and manipulation tasks, highlighting 3DGS as a practical bridge between rich perception and real-time motion planning.

I. INTRODUCTION

Reducing visual information into primitive geometrical abstractions has been an effective approach to systematically isolating perception and planning subsystems in robotics. This separation, however, comes at the expense of information bottlenecks and reduced robustness. 3D Gaussian Splatting (3DGS) has recently emerged as an intermediate representation that preserves rich visual information while explicitly capturing the 3D geometry of the scene. In fact, this representation and its rasterizer can be regarded as an action-conditioned world model: given a candidate future robot pose, the rasterizer predicts the corresponding visual observation at a fraction of the cost of large learned world models [1], [2].

Motivated by this potential, we introduce *CollisionSplatting*, a highly efficient and modular distance/collision metric that operates directly on the standard 3DGS representation and can be seamlessly incorporated into standard motion planning and control algorithms as a single-kernel GPU module. We show that, when combined with visual reward models, our metric enables real-time collision-aware planning while respecting semantic objectives imposed by the reward model.

Most existing work proposes distance metrics that are tightly coupled to a particular planning algorithm or robot model. Splat-Nav [3] was among the earliest and introduced a robot-Gaussian binary collision classifier as the core of a safe-flight-corridor-based planning system. Splat-Nav replaces each Gaussian in 3DGS with an equivalent probability-level-set ellipsoid and computes its collision state with the robot, which is likewise modeled as an ellipsoid. SAFER-SPLAT [4] also considers each Gaussian as an ellipsoid, but instead of a binary check, quantifies the distance between the robot (spherical) and each Gaussian via a constrained numerical optimization problem on top of which a Control Barrier Function (CBF) safety filter is formulated.

While Splat-Nav and SAFER-SPLAT replace Gaussians with deterministic collision primitives, ATLASNav [5] adopts a probabilistic view, modeling both the robot and Gaussians as spatial probability distributions. To achieve analytical tractability, ATLASNav assumes isotropic 3DGS Gaussians, spherical robot geometry, and approximately homogeneous scene density. Finally, SPLANNING [6] introduces a statistically rigorous, risk-aware collision-avoidance framework for 3DGS-based manipulation. However, to guarantee probabilistic consistency, SPLANNING relies on a custom *Normalized 3DGS* reconstruction pipeline, meaning its theoretical guarantees apply to that modified representation rather than to standard unnormalized 3DGS datasets [7] or existing generative models [8].

In contrast, our work proposes a simple distance/collision metric inspired by a probability-propagation scheme. It operates directly on standard 3DGS reconstructions and accommodates both anisotropic and isotropic Gaussians, disk-like 2DGS Gaussians [9], and ellipsoidal or spherical multi-link robot primitives. Its simplicity and streaming feed-forward computation graph make our metric highly parallelizable, with per-splat-localized memory requirements that avoid large intermediate allocations or multi-stage GPU/CPU data transfers. Consequently, our method achieves significant memory efficiency and high collision-checking throughput while maintaining on-par or better performance compared to baselines.

Furthermore, we show how the efficiency of our metric, combined with the efficient 3DGS rasterizer, enables real-time joint visual-geometric planning under a Model Predictive Path Integral (MPPI) formulation for both a 7-DoF manipulator and a quadruped mobile robot. This MPPI integration example is closely related to BEINGS [10], which formulates image-goal navigation in image space, but our

¹ Machines in Motion Lab, Tandon School of Engineering, New York University

² Artificial and Natural Intelligence Toulouse Institute (ANITI)

³ Department of Smart Systems Technologies, University of Klagenfurt, Austria.

Code is available at <https://github.com/machines-in-motion/CollisionSplatting>.

approach explicitly penalizes collisions in real time using a single-kernel GPU-accelerated collision cost.

The paper is organized as follows. Sec. II reviews the 3DGS background and presents our distance metric, Sec. III covers baseline comparisons and ablations, and Sec. IV presents real-world deployments before concluding in Sec. V.

II. METHOD

A. Background

Gaussian splatting may be thought of as an extension of point-clouds. It represents the 3D scene as a set of Gaussian primitives (referred to as splats in this paper) $\mathcal{S} = \{\mathcal{G}_i\}$ with locations and spans parameterized by translation vectors $\boldsymbol{\mu}_i^g \in \mathbb{R}^3$ and covariance matrices $\boldsymbol{\Sigma}_i^g \in \mathbb{R}^{3 \times 3}$:

$$\mathcal{G}_i(\mathbf{p}) = \exp\left(-\frac{1}{2}(\mathbf{p} - \boldsymbol{\mu}_i^g)^\top \boldsymbol{\Sigma}_i^{g-1}(\mathbf{p} - \boldsymbol{\mu}_i^g)\right), \quad (1)$$

where $\mathbf{p} \in \mathbb{R}^3$ is the splat's position in the world frame $\mathcal{W}$. Each covariance $\boldsymbol{\Sigma}_i^g$ is parametrized by a diagonal matrix $\mathbf{S}_i$ describing the scale of each principal axis in the Gaussian's local frame $\mathcal{G}_i$ and a rotation matrix ${}^{\mathcal{W}}\mathbf{R}_{\mathcal{G}_i}$ that represents the orientation of this frame with respect to the global world frame $\mathcal{W}$:

$$\boldsymbol{\Sigma}_i^g = {}^{\mathcal{W}}\mathbf{R}_{\mathcal{G}_i} \mathbf{S}_i \mathbf{S}_i^\top {}^{\mathcal{W}}\mathbf{R}_{\mathcal{G}_i}^\top. \quad (2)$$

The differentiable rendering process includes transforming each 3D Gaussian into the camera frame $\mathcal{C}$ and then projecting it onto the image plane via a local affine map $\mathbf{J}$ [11]:

$$\boldsymbol{\Sigma}'_i = \mathbf{J} {}^{\mathcal{C}}\mathbf{R}_{\mathcal{G}_i} \boldsymbol{\Sigma}_i^g {}^{\mathcal{C}}\mathbf{R}_{\mathcal{G}_i}^\top \mathbf{J}^\top. \quad (3)$$

The first two rows and columns of $\boldsymbol{\Sigma}'_i$ yield the 2D covariance of the projected 3D Gaussian $\mathcal{G}_i$ onto the image plane. The projection of the 3D means $\boldsymbol{\mu}_i^g$ follows the conventional point-based projection mapping $\boldsymbol{\mu}' = \pi(\boldsymbol{\mu}_i^g, {}^{\mathcal{C}}\mathbf{T}_{\mathcal{G}_i}, \mathbf{K}_{\text{cam}})$, where ${}^{\mathcal{C}}\mathbf{T}_{\mathcal{G}_i} \in \mathbb{R}^{4 \times 4}$ is the homogeneous transformation from Gaussian frame $\mathcal{G}_i$ to camera frame $\mathcal{C}$ and $\mathbf{K}_{\text{cam}} \in \mathbb{R}^{3 \times 3}$ is the camera intrinsic matrix.

Given these 2D projections of the 3D Gaussians on the image plane, the color at point $\mathbf{x}$ in image space is computed through volumetric alpha blending of the depth-wise sorted Gaussian primitives along the ray passing through $\mathbf{x}$:

$$\mathbf{c}(\mathbf{x}) = \sum_{k=1}^K c_k \alpha_k \mathcal{G}_k^{2D}(\mathbf{x}) \prod_{j=1}^{k-1} (1 - \alpha_j \mathcal{G}_j^{2D}(\mathbf{x})), \quad (4)$$

where c_k and α_k are learnable parameters representing the color and opacity of Gaussian $\mathcal{G}_k$, and $\mathcal{G}_k^{2D}(\mathbf{x})$ is the 2D counterpart of (1) parametrized by $\boldsymbol{\Sigma}'_i, \boldsymbol{\mu}'_i$ in image space. The term $\prod_{j=1}^{k-1} (1 - \alpha_j \mathcal{G}_j^{2D}(\mathbf{x}))$ indicates how much light is occluded by other Gaussian primitives between the camera and Gaussian $\mathcal{G}_k$.

During reconstruction, the scene is initialized randomly or from SfM/RGB-D priors, then optimized by alternating densification and parameter updates to minimize the L1 rendering loss. Each Gaussian primitive may be augmented with any d -dimensional task-dependent feature vector (similar to color). During rendering, these features are projected into the

image space, creating novel-view feature maps. Depth maps can be similarly rendered by replacing the color in (4) with the splat's distance from the camera.

B. CollisionSplatting Distance Metric

We assume that the robot may be decomposed into a set of collision ellipsoids. Compared to previous works that assume spherical representations [4], [5], [12], our choice allows for a greater level of expressivity without trading off computational efficiency.

1) *Distance Metric Between a Point and an Ellipsoid:* Consider the robot as a collection of ellipsoidal bodies $\mathcal{E}_j$, each expressed as the $s_{ij}^2 = 1$ level set:

$$(\mathbf{X} - \boldsymbol{\mu}_j^r)^\top \mathbf{A}_j (\mathbf{X} - \boldsymbol{\mu}_j^r) = s_{ij}^2, \quad (5)$$

where $\boldsymbol{\mu}_j^r \in \mathbb{R}^3$ denotes the center of the ellipsoidal link in the world frame and $\mathbf{A}_j$ encodes its span and orientation. The matrix $\mathbf{A}_j$ may be decomposed as $\mathbf{A}_j = {}^{\mathcal{W}}\mathbf{R}_{\mathcal{E}_j} \mathbf{P}_j {}^{\mathcal{W}}\mathbf{R}_{\mathcal{E}_j}^\top$, where ${}^{\mathcal{W}}\mathbf{R}_{\mathcal{E}_j}$ is the orientation of the ellipsoid body frame $\mathcal{E}_j$ with respect to the world frame, and

$$\mathbf{P}_j = \begin{bmatrix} 1/a_j^2 & 0 & 0 \\ 0 & 1/b_j^2 & 0 \\ 0 & 0 & 1/c_j^2 \end{bmatrix}, \quad (6)$$

with $a_j, b_j, c_j \in \mathbb{R}_+$ denoting the lengths of each semi-axis of $\mathcal{E}_j$ in its body frame. Now consider $\boldsymbol{\mu}_i^g \in \mathbb{R}^3$ as the position of a point obstacle in the world frame. Substituting $\mathbf{X} = \boldsymbol{\mu}_i^g$ in (5) yields s_{ij}^2 with three possible cases: if the point is on the surface of the body, $s_{ij}^2 = 1$; if it is inside, $s_{ij}^2 < 1$; and if the point is outside (not colliding with the ellipsoid), $s_{ij}^2 > 1$. This measure, often referred to as a scaling-function distance, is employed in [13] to formulate a GPU-accelerated CBF-based framework using LiDAR point clouds.

2) *Distance Metric Between a Gaussian and an Ellipsoid:* We now extend the point-based metric in (5) to work with Gaussian obstacles. We consider each Gaussian $\mathcal{G}_i$ as a probability density over the possible positions of a point obstacle $\mathbf{p}_i$, with $\mathbf{p}_i \sim \mathcal{N}(\boldsymbol{\mu}_i^g, \boldsymbol{\Sigma}_i^g)$.

Drawing inspiration from the uncertainty-propagation view of the 3DGS projection in Eq. 3, we treat the point-based distance function $s_{ij}^2(\mathbf{X})$ as a nonlinear transformation and adopt a linearization-based covariance propagation scheme to map input uncertainty (3DGS scale) into uncertainty over the distance. Specifically, we linearize $s_{ij}^2(\mathbf{X})$ around $\mathbf{X} = \boldsymbol{\mu}_i^g$ and propagate the input uncertainty (point obstacle location) to yield a 1D Gaussian distribution $s_{ij}^2 \sim \mathcal{N}(\mu_{ij}^s, \sigma_{ij}^2)$, where $\mu_{ij}^s = s_{ij}^2(\boldsymbol{\mu}_i^g)$ and σ_{ij}^2 is computed as:

$$\sigma_{ij}^2 = \left(\left. \frac{\partial s_{ij}^2}{\partial \mathbf{X}} \right|_{\mathbf{X}=\boldsymbol{\mu}_i^g} \right) \boldsymbol{\Sigma}_i^g \left(\left. \frac{\partial s_{ij}^2}{\partial \mathbf{X}} \right|_{\mathbf{X}=\boldsymbol{\mu}_i^g} \right)^\top. \quad (7)$$

This propagation is accurate when the nonlinearity of $s_{ij}^2(\cdot)$ is moderate in a neighborhood of $\boldsymbol{\mu}_i^g$ and the Gaussian scale is small relative to the robot-obstacle distance. While this assumption empirically holds in well-reconstructed 3DGS

scenes, in Sec. III-C.2 we show that even when it is violated, our metric becomes more conservative, which is a desirable failure mode for safety-critical planning.

We then define our conservative distance metric as

$$\tilde{s}_{ij}^2 = \max(0, s_{ij}^2 - n_\sigma \sqrt{\sigma_{ij}^2}), \quad (8)$$

where $n_\sigma \geq 0$ controls the level of conservatism and can be set based on a desired confidence level. Setting $n_\sigma = 0$ recovers the original point-based distance, implying that the same planning pipeline can be seamlessly used with point clouds. Finally, to determine the collision state of a robot link, we adopt a numerically stable recursive softmin operation over the splats. Note that the complexity of evaluating the collision state between a K -link robot and a Gaussian scene with N primitives is $O(KN)$. In practice the robot has a limited reach at each instant, so we adopt a GPU-accelerated spatial hash grid to filter out unreachable parts of the map.

C. Assumptions and Design Choices

Our formulation is probability-inspired rather than statistically calibrated: the propagated variance in (7) is treated as an uncalibrated uncertainty measure. Furthermore, we aggregate per-splat distances using local operations (recursive softmin over neighbors), which keeps computations localized and avoids global normalization or coupling between Gaussians, thereby reducing memory footprint and runtime. Proper uncertainty calibration of the aggregated distance metric remains an interesting direction for future work. In addition, 3DGS scenes may represent transparent or reflective objects using mixtures of low-opacity splats away from the true surface. For collision checking, we assume obstacles of interest are not highly transparent and discard splats that are both very low opacity and large in spatial extent to filter floaters. Modern 3DGS variants such as 2DGS [9] tend to reduce such artifacts, further mitigating the need for this heuristic.

D. MPPI Controller With Image-Based Cost

We adopt a standard MPPI formulation for receding-horizon control. At each iteration, we sample K control sequences around a nominal sequence $\bar{U}$, roll them out in parallel under the system dynamics, and evaluate the resulting trajectories with a cost

$$C^k = C_{\text{collision}}^k + C_{\text{task}}^k + C_{\text{image}}^k + C_{\text{control}}^k, \quad (9)$$

where C_{task}^k encodes any geometric tracking objectives, C_{control}^k penalizes control effort, $C_{\text{collision}}^k$ is the collision avoidance cost, and C_{image}^k represents any visual cost function. The trajectory costs are converted into weights via a softmin with temperature λ :

$$w^k = \frac{\exp(-C^k/\lambda)}{\sum_{l=1}^K \exp(-C^l/\lambda)}, \quad (10)$$

and these weights are used to update the nominal sequence $\bar{U}$ in the direction of lower-cost perturbations. After a small, fixed number of iterations, the first control in the updated

sequence is applied to the robot, and $\bar{U}$ is warm-started at the next MPC step by temporally shifting the previous solution. In practice, we parameterize the control sequence by B-spline knots to ensure control signal smoothness [14].

1) *Collision Cost*: We use our collision metric in (8) to define a collision cost along each trajectory. For each robot link j and Gaussian i , we define

$$d_{ij} = d_{\max} \exp(-\gamma \max(\tilde{s}_{ij}^2 - s_{\min}, 0)), \quad (11)$$

where $\gamma \in \mathbb{R}_+$ is a user-defined decay factor that reduces the influence of distant splats, $d_{\max}$ is the maximum cost per splat, and $s_{\min}$ is the minimum allowable scaling factor (set to one in our case). At each time step, these per-splat, per-link values are averaged over neighboring splats (via spatial hashing) and summed over links to yield a running collision cost. The trajectory collision cost $C_{\text{collision}}^k$ is then computed as the sum of this running cost over the horizon. Our GPU implementation evaluates this term on batches of shape (K, T, N_{link}) , where K is the number of rollouts, T the prediction horizon, and N_{link} the number of ellipsoidal primitives describing the robot.

2) *Semantic Image-Based Cost*: Gaussian splatting can efficiently render images from arbitrary viewpoints in parallel [15]. We leverage this capability to add a terminal visual cost. Specifically, the terminal state of each trajectory is first fed to the 3DGS rasterizer to synthesize an image. These images are then passed through a visual encoder to evaluate a learned reward. In this paper, we formulate this reward as the squared Euclidean distance in the embedding space of a foundation visual encoder (DINOv2 in our experiments):

$$\|\text{enc}(\mathbf{I}_{\text{target}}) - \text{enc}(\mathbf{I}_G)\|_2^2, \quad (12)$$

where $\text{enc}(\cdot)$ denotes the image encoder, $\mathbf{I}_{\text{target}}$ the goal image, and $\mathbf{I}_G$ the image generated from the terminal state of the trajectory.

E. RRT Global Planner

We further demonstrate the applicability of our distance metric in global motion planning by implementing a batched GPU-accelerated Rapidly-Exploring Random Tree (RRT) algorithm. The collision metric in Eq. 8 is used as a parallelized collision-checking engine for all candidate edges in each batch. Specifically, we compute the distance metric s_{ij}^2 between each robot link and all splats, marking a state as in collision if any link has more than a small user-defined number of Gaussians with $s_{ij}^2 < 1$.

The RRT grows in parallel on the GPU. At each iteration, N_{batch} random samples are drawn from the configuration space, and for each, the closest tree node is expanded toward the sample up to a maximum stride length. Each edge is checked for collisions at discrete points defined by a small user-specified resolution, and collision-free edges are appended as new nodes.

All tree data structures and buffers reside in GPU memory to minimize data transfer. Collision checking is implemented as a kernel operating on batches of shape $N_{\text{batch}} \times N_{\text{strides}} \times N_{\text{links}}$, where N_{strides} denotes the number of checks per edge based on the chosen resolution.

III. COLLISIONSPLATTING METRIC EVALUATION

A. Setup

We conduct our study here using the Stonehenge scene adopted by Splat-Nav [3]. It features various gate-like structures and columns with varying opening widths (Fig. 2), thereby representing different degrees of navigation difficulty. The scene scale is normalized to 1, and the robot is modeled as a single sphere of radius 0.015 (i.e., 1.5% of the scene scale) for the baseline metric evaluations. We use NVIDIA Warp [16] to implement our custom kernels and execute all simulated studies on a workstation equipped with an NVIDIA RTX 6000 Pro GPU and an AMD Ryzen 9 9950X CPU.

B. Baselines Comparison

In this section, we compare our distance metric against representative baselines in terms of collision classification performance and computational throughput. As representative of methods modeling Gaussians as elliptical collision primitives, we choose Splat-Nav [3] and SAFER-SPLAT [4]. Additionally, we compare our metric against ATLASNav [5], a closely related work that derives a probabilistic distance metric for isotropic Gaussian scenes and spherical robots (ours operates on the more general case of anisotropic 3DGS scenes and ellipsoidal robots). We also include SPLANNING [6], which rigorously upper-bounds the probability of collision between a robot (represented as a collection of spheres) and a radiance field by computing the volume integral of a normalized 3DGS density. Since SPLANNING’s normalized reconstruction pipeline was not open-sourced at the time of writing, we evaluate its metric on our standard 3DGS reconstruction; because the precision-recall curve sweeps the threshold from 0 to 1 rather than using absolute probabilities, this has minimal effect on fairness. Furthermore, note that we focus on the distance/collision metric proposed for each baseline in this section and do not consider the planner choices to maintain comparability.

We evaluate each distance metric on a grid of points in the Stonehenge environment ($\approx 200\,000$ Gaussians splats) and show the precision-recall curves for each method, along with the computational throughput and GPU RAM utilization in Fig. 1 and Table I, respectively. We see that our metric performs on par with Splat-Nav, SAFER-SPLAT, and SPLANNING, while outperforming ATLASNav. We attribute ATLASNav’s lower performance to its isotropic Gaussian assumption. In our study, we accommodate this need by replacing the anisotropic Gaussians with spherical counterparts through scale averaging.

In terms of computational efficiency, Table I reveals that while ATLASNav and SPLANNING implementations support batched GPU evaluation, their reliance on large intermediate tensor allocations results in significant memory consumption for applications where batched collision checking is essential (e.g., zeroth-order optimization and MPPI). In our experiment, these methods respectively required 55.0 GB and 14.5 GB of GPU memory for a batch size of 4,096.

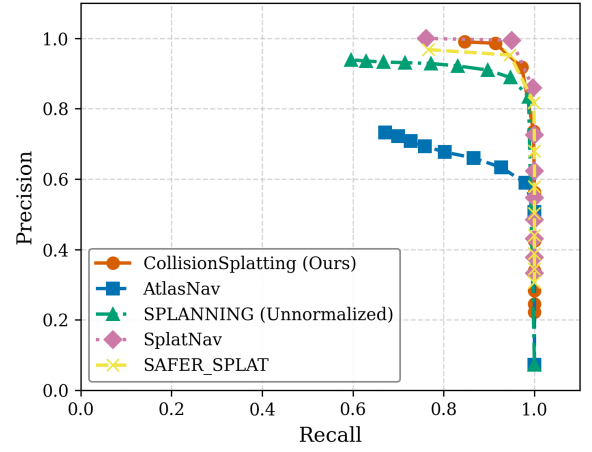


Fig. 1. Precision-recall curves for CollisionSplatting distance metric and representative baselines.

In contrast, our CollisionSplatting metric evaluates collision states using a direct, single-pass, per-splat propagation technique that entirely avoids intermediate allocations. In our experiment, this streaming formulation only required 0.16 GB of VRAM for the same batch size and scales to larger batch sizes without a significant increase in memory requirements (we report batch size of 32,768 with a negligible VRAM increase to 0.166 GB). This memory efficiency and localized in-kernel computation model enable significantly higher, more scalable throughput for our method, which directly enables our real-time experiments.

C. Ablation Study

We implement a GPU-accelerated RRT planner that leverages the efficiency and speed of our collision metric to grow the tree 32 nodes at a time. We further optimize the global path from RRT by implementing a zeroth-order optimizer that minimizes path length while penalizing collisions (a single MPPI step, as described in Sec.II-D, with a horizon set to the full plan length). We evaluate this planner in the Stonehenge scene and on a set of randomized trials, in which the robot starts at the center and navigates to a randomly selected goal along the scene’s border.

1) *Conservatism Factor*: Table II reports the dimensionless path length (normalized scene), the minimum distance to the closest Gaussian mean $d_{\min}$ (a proxy for safety), and normalized path smoothness for different conservatism factor n_{σ} values before and after refinement. Note that when $n_{\sigma} = 0$, the uncertainty propagation is nullified, and the scene effectively reduces to a dense point-cloud of Gaussian means, implying that our metric may also be seamlessly applied to point-cloud maps. We can see that as n_{σ} increases, both the path length and the minimum obstacle distance grow, indicating more conservative plans. This effect is also shown qualitatively in Fig. 2, where larger n_{σ} values cause the planner to avoid narrow gates and navigate around obstacles instead. Notably, after refinement, obstacle-avoiding clearances increase while the path length

TABLE I
COLLISION DETECTION THROUGHPUT AND VRAM UTILIZATION COMPARISON

Method	Throughput (GPU) (↑) Checks/s (mean ± STD)	Throughput (CPU) (↑) Checks/s (mean ± STD)	Acceleration (↑) (n _{GPU} /n _{CPU})	VRAM Utilization (↓) (GB)	Batch Size
SplatNav [3]	162.1 ± 1.6	23.7 ± 0.5	6.84×	0.22	1
SAFER-SPLAT [4]	217.5 ± 3.4	34.4 ± 0.5	6.33×	0.38	1
ATLASNav [†] [5]	16,318.7 ± 381.0	262.2 ± 17.3	62.23×	55.0	4,096
SPLANNING [6]	157,538.5 ± 603.6	2,409.4 ± 18.3	65.38×	14.50	4,096
CollisionSplatting (Ours)	215,578.9 ± 7,660.2 1,638,400.0 ± 32,125.5	13,653.3 ± 440.4 12,750.2 ± 88.7	15.7×	0.16 0.166	4,096 32,768 (4,096 × 8)

[†] Evaluated using our own implementation, as the official source code for ATLASNav was not publicly available at the time of writing.

decreases, and since optimization occurs directly over B-spline knots, the resulting paths are also smooth by design.

2) *Linearization Effect*: As discussed in Sec. II-B, uncertainty propagation via linearization in Eq. 7 is accurate when Gaussian scales are sufficiently smaller than the robot size (i.e., the minimum distance). In this section, we study our metric’s response as the Gaussian-to-robot scale ratio increases. According to Eq. 7, the propagated scaling uncertainty grows proportionally with the Gaussian scale Σ_i . Consequently, in poorly observed or uncertain regions characterized by large Gaussians, our metric inherently overestimates the collision risk. This is a highly desirable property, as it encourages more conservative behavior in regions of the map that are more uncertain. To validate this, we train five 3DGS Stonehenge models with increasing densification thresholds to generate models with increasing Gaussian sizes (M1–M5). First, we compute our distance metric along a straight collision-free path and plot the output for each model in Fig. 3. As shown, for scenes with larger Gaussians, our distance metric produces smaller values, indicating more conservativeness. Next, we run the planner discussed earlier on three different scenes and report the result in Table III. As expected, we observe that the planner operates more conservatively in scenes with larger Gaussians.

IV. REAL-WORLD DEMONSTRATIONS

A. Joint Visual-Geometric Planning in Manipulation

Our efficient distance metric, along with the real-time rendering capability of the 3DGS rasterizer, enables us to formulate control problems in which geometric terms, such as collision avoidance, are jointly optimized with image-based objectives. In this section, we demonstrate one example of such a formulation in a visual goal-reaching manipulation task.

1) *Task and Setup*: We consider a Flexiv Rizon10s 7-DoF manipulator robot equipped with a wrist-mounted camera and tasked with reaching a desired target image while avoiding collisions with surrounding obstacles. The experimental setup is shown in Fig. 4(a).

2) *Formulation*: We implement the MPPI controller described in Sec. II-D for our robotic arm. Each link of the arm is encapsulated by an ellipsoidal primitive, whose poses are computed over the planning horizon using a custom GPU-accelerated forward kinematics Warp kernel. At each control

update, a batch of images is synthesized for the terminal state of each MPPI rollout via the 3DGS rasterizer. These images are fed to the visual reward model to compute the semantic cost (in the DINOv2 embedding space in our example). After a fixed number of updates, the first control output is sent to the robot, and the process is repeated with a warm start from the shifted solution from the previous step. Note that our highly efficient collision metric is essential for this example because while reactive collision avoidance may not require long-horizon MPC, proper long-horizon exploration is essential for optimizing the terminal visual reward without getting trapped in local minima.

3) *Result*: Table IV reports the reaching accuracy across five randomized goal images. Notably, optimizing purely for visual reaching accuracy without the collision cost results in paths that physically intersect the environment, leading to a 100% collision rate with the box walls. Thus the unconstrained system’s lower translation errors are physically unexecutable. Enabling our CollisionSplatting cost keeps a positive clearance (0.017 m), trading slight visual precision for collision-free execution. The planner is then deployed on the real robot, where the MPPI operates at 5 Hz over a receding horizon, and a subset of the resulting trajectory is executed by a compliant impedance controller at 1000 Hz. As qualitatively shown in Fig. 4(a), the robot successfully reaches the goal image while avoiding contact with the environment.

B. Image-Goal-Based Navigation

In this example, we integrate the semantic DINOv2-based cost and our distance metric to formulate a dual-stage RRT+MPPI planner that enables collision-aware path planning toward visually specified targets.

1) *Task and Setup*: In this task, a mobile robot (Unitree-Go2) with an internal 3DGS representation of the world is provided with a target image and must navigate to a semantically similar object within the environment.

2) *Formulation*: Instead of searching over the entire space randomly to find an object, we leverage our distance metric to execute a fast batched-RRT in exploration mode (without a predefined geometric target). This will rapidly generate a collection of collision-free robot configurations along with the paths to reach them. This greatly reduces the search space during image retrieval and ensures a feasible path from the current state to the retrieved image. During the retrieval

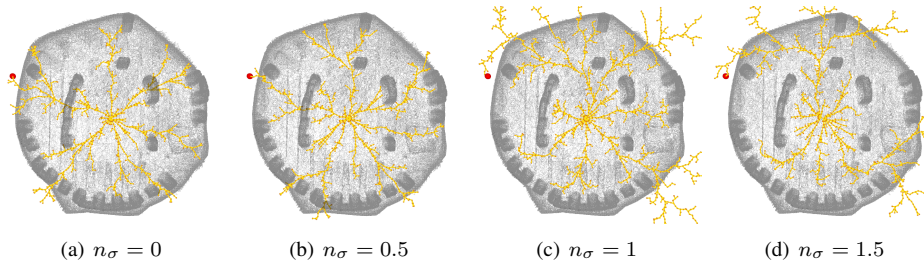


Fig. 2. Effect of the conservatism factor n_σ on the paths discovered by the RRT planner. Increasing n_σ yields more conservative trajectories that avoid tight passages.

TABLE II
GLOBAL RRT PLANNING AND LOCAL MPPI REFINEMENT VS VARIOUS CONSERVATISM FACTORS.

n_σ (conservatism)	Path Length ($\downarrow$) (RRT)	Path Length ($\downarrow$) (RRT+Refinement)	d_{min} ($\uparrow$) (RRT)	d_{min} ($\uparrow$) (RRT+Refinement)	Smoothness ($\downarrow$) (RRT)	Smoothness ($\downarrow$) (RRT+Refinement)
0.00	0.6017 ± 0.0841	0.5748 ± 0.0778	0.0236 ± 0.0114	0.0300 ± 0.0135	0.0124 ± 0.0051	0.0003 ± 0.0003
0.25	0.5818 ± 0.0723	0.5477 ± 0.0599	0.0252 ± 0.0121	0.0345 ± 0.0118	0.0133 ± 0.0049	0.0003 ± 0.0002
0.50	0.5685 ± 0.0580	0.5441 ± 0.0465	0.0225 ± 0.0086	0.0311 ± 0.0115	0.0136 ± 0.0058	0.0004 ± 0.0003
0.75	0.6390 ± 0.1773	0.5875 ± 0.1298	0.0267 ± 0.0112	0.0345 ± 0.0133	0.0134 ± 0.0047	0.0003 ± 0.0003
1.00	0.6048 ± 0.0806	0.5590 ± 0.0636	0.0301 ± 0.0122	0.0360 ± 0.0141	0.0151 ± 0.0058	0.0003 ± 0.0003
1.25	0.6643 ± 0.1626	0.6084 ± 0.1160	0.0286 ± 0.0112	0.0334 ± 0.0165	0.0158 ± 0.0074	0.0003 ± 0.0003
1.50	0.6772 ± 0.1912	0.6275 ± 0.1575	0.0330 ± 0.0100	0.0378 ± 0.0142	0.0136 ± 0.0074	0.0003 ± 0.0003
1.75	0.7822 ± 0.2598	0.7031 ± 0.2138	0.0358 ± 0.0072	0.0440 ± 0.0073	0.0173 ± 0.0066	0.0002 ± 0.0002
2.00	0.7721 ± 0.2593	0.7072 ± 0.2272	0.0392 ± 0.0093	0.0459 ± 0.0091	0.0147 ± 0.0057	0.0002 ± 0.0002

phase, we synthesize the expected image observation for each node in the tree using the 3DGS rasterizer and compute the DINO feature embedding. Given the target image embedding, we select a semantically similar node in the tree. Finally, we use the MPPI formulation discussed earlier to control the robot along the path while avoiding obstacles on the fly.

3) *Result:* As illustrated in Fig. 4(b), the accelerated RRT plus the DINO distance enables collision-free path retrieval based on generic object goal-images, even when their visual appearances differ from the real scene. As shown in Fig. 4(c), this path serves as a reference for our collision-aware MPPI controller. The controller then optimizes a shorter horizon of command velocities and sends the first command to the robot’s onboard locomotion policy at each MPC step. Notably, in this experiment, we provided only the final goal position found by the RRT to the MPPI controller, discarding intermediate waypoints. This serves to stress-test the controller’s collision-avoidance capability without dense geometric guidance from the global plan. Because our distance metric is highly efficient, we were able to extend the MPPI prediction horizon long enough to autonomously navigate around obstacles while maintaining real-time control frequencies.

TABLE III

PLANNING CONSERVATISM VS SCENES WITH INCREASINGLY LARGER GAUSSIAN PRIMITIVES.

Scene	Path Length ($\downarrow$)	Closest Distance ($\uparrow$)
M_1 (Smallest Gaussians)	0.6048 ± 0.0806	0.030 ± 0.012
M_2	0.6747 ± 0.1721	0.031 ± 0.014
M_3 (Largest Gaussians)	0.7531 ± 0.2812	0.035 ± 0.013

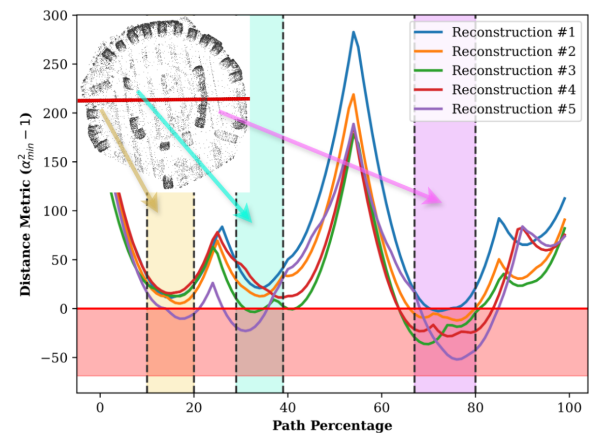
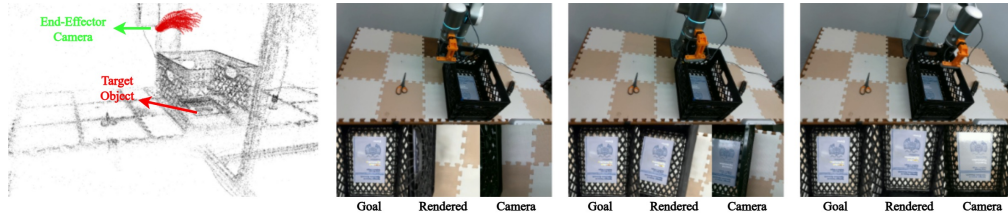


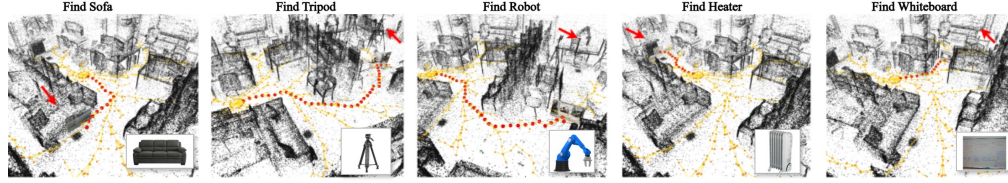
Fig. 3. Computed collision distance along a linear path crossing reconstructed scenes with different Gaussian scales. As the distribution of large Gaussians increases, the metric yields smaller (more conservative) distance values, especially at sections of the path that traverse across obstacles.

V. CONCLUSION

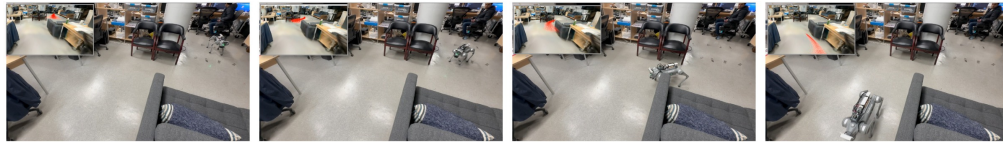
We presented CollisionSplatting, a GPU-friendly probabilistic distance metric for collision-aware planning directly in standard 3DGS scenes. The metric propagates each Gaussian’s spatial uncertainty through an ellipsoid-based scaling-function distance, exposing a single tunable conservatism parameter. Owing to its feed-forward, streaming structure, we integrated it as a batched collision-checking primitive in both a GPU-accelerated RRT and an MPPI controller, jointly optimizing geometric safety and image-conditioned objectives via rasterized 3DGS observations and DINO embeddings. In simulation, our metric matched or exceeded Splat-



(a) Collision-aware visual target reaching.



(b) Collision-free path finding to image-defined objects.



(c) Collision-aware goal navigation.

Fig. 4. CollisionSplatting deployed to real-world locomotion and manipulation for collision-aware visual reaching and object navigation tasks.

TABLE IV

VISUAL TARGET REACHING PERFORMANCE WITH OR WITHOUT COLLISION COST.

Configuration	Distance to Closest Splat (m $\uparrow$)	Translation Error (m $\downarrow$)	Rotation Error (degrees $\downarrow$)
Without Collision Cost	$-0.0212 \pm 0.029^\dagger$	0.0147 $\pm$ 0.0057	3.4806 $\pm$ 1.0627
With Collision Cost	0.017 $\pm$ 0.008	0.031 $\pm$ 0.014	5.693 $\pm$ 1.503

† A negative distance indicates physical collision with the environment.

Nav, SAFER-SPLAT, ATLASNav, and SPLANNING while substantially improving throughput and VRAM use. On real robots, we demonstrated collision-aware visual reaching and image-goal navigation, showing 3DGS with CollisionSplatting as a practical interface between rich perception and real-time control.

REFERENCES

- [1] A. Bar, G. Zhou, D. Tran, T. Darrell, and Y. LeCun, "Navigation world models," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 15 791–15 801.
- [2] G. Lu, B. Jia, P. Li, Y. Chen, Z. Wang, Y. Tang, and S. Huang, "Gwm: Towards scalable gaussian world models for robotic manipulation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 9263–9274.
- [3] T. Chen, O. Shorinwa, W. Zeng, J. Bruno, P. M. Dames, and M. Schwager, "Splat-nav: Safe real-time robot navigation in gaussian splatting maps," *CoRR*, vol. abs/2403.02751, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2403.02751>
- [4] T. Chen, A. Swann, J. Yu, O. Shorinwa, R. Murai, M. Kennedy III, and M. Schwager, "Safer-splat: A control barrier function for safe navigation with online gaussian splatting maps," *arXiv preprint arXiv:2409.09868*, 2024.
- [5] D. Ong, Y. Tao, V. Murali, I. Spasojevic, V. Kumar, and P. Chaudhari, "Atlas navigator: Active task-driven language-embedded gaussian splatting," *arXiv preprint arXiv:2502.20386*, 2025.
- [6] J. Michaux, S. Isaacson, C. E. Adu, A. Li, R. K. Swayampakula, P. Ewen, S. Rice, K. A. Skinner, and R. Vasudevan, "Let's make a splat: Risk-aware trajectory optimization in a normalized gaussian splat," *IEEE Transactions on Robotics*, 2025.
- [7] M. T. I. SpatialVerse Research Team, "Interiorgs: A 3D gaussian splatting dataset of semantically labeled indoor scenes," <https://huggingface.co/datasets/spatialverse/InteriorGS>, 2025.
- [8] World Labs, "World labs," <https://www.worldlabs.ai/>, accessed: 2026-03-04.
- [9] B. Huang, Z. Yu, A. Chen, A. Geiger, and S. Gao, "2D gaussian splatting for geometrically accurate radiance fields," in *SIGGRAPH 2024 Conference Papers*. Association for Computing Machinery, 2024.
- [10] W. Meng, T. Wu, H. Yin, and F. Zhang, "Beings: Bayesian embodied image-goal navigation with gaussian splatting," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 5252–5258.
- [11] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139–1, 2023.
- [12] R. Jin, Y. Gao, Y. Wang, Y. Wu, H. Lu, C. Xu, and F. Gao, "Gs-planner: A gaussian-splatting-based planning framework for active high-fidelity reconstruction," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 11 202–11 209.
- [13] B. Dai, R. Khorrambakht, P. Krishnamurthy, and F. Khorrami, "Sailing through point clouds: Safe navigation using point cloud based control barrier functions," *IEEE Robotics and Automation Letters*, 2024.
- [14] T. Howell, N. Gileadi, S. Tunyasuvunakool, K. Zakka, T. Erez, and Y. Tassa, "Predictive Sampling: Real-time Behaviour Synthesis with MuJoCo," <https://arxiv.org/abs/2212.00541>, dec 2022.
- [15] V. Ye, R. Li, J. Kerr, M. Turkulainen, B. Yi, Z. Pan, O. Seiskari, J. Ye, J. Hu, M. Tancik, and A. Kanazawa, "gsplat: An open-source library for Gaussian splatting," *arXiv preprint arXiv:2409.06765*, 2024. [Online]. Available: <https://arxiv.org/abs/2409.06765>
- [16] M. Macklin, "Warp: A high-performance python framework for gpu simulation and graphics," <https://github.com/NVIDIA/warp>, March 2022, NVIDIA GPU Technology Conference (GTC).